

INTERNATIONAL CYBERSECURITY & DIGITAL FORENSICS

ACADEMY (ICDFA)

DIGITAL FORENSICS LABORATORY REPORT

Data Carving and File Recovery from Raw Disk and USB Images

Laboratory Assessment - Data Carving and File Recovery Techniques

Prepared by: Nebeuwa Ifeanyichukwu Raphael

Role: Junior Digital Forensic Analyst

Course Code: CIP-B102

Examination Platform: Kali Linux (kali@kali)

Submission Date: 5 September 2026

1. Executive Summary

This report documents my performance of an ICDDFA Module 7 laboratory assessment focused on data carving and file recovery. Acting in the role of a junior digital forensic analyst, I examined a raw JPEG sample for header/footer signatures and embedded metadata, performed direct data-unit examination of a raw DD floppy image using The Sleuth Kit, identified embedded file structures with Binwalk, and carved deleted JPEG files from a raw USB image using Scalpel.

Prior to beginning technical analysis, I reviewed the Module 7 lecture material on data carving and file recovery, which included worked examples for each of the required tools. I encountered a genuine acquisition obstacle at the outset: the authorised Dropbox-hosted evidence files could not be retrieved via standard command-line tools (wget) or their in-browser preview, due to Dropbox's file-type preview limitations and its shareable-link authentication flow. I resolved this by cross-referencing the Module 7 lecture slides, which contained direct, authorised GitHub-hosted download URLs for the same evidence files, and used those instead.

All six parts of the lab (A through F) were completed. Key results include: a lossless hex-dump reconstruction of the sample JPEG verified by matching hashes; recovery of rich embedded EXIF/XMP metadata revealing the file's edit history and true provenance; successful inode- and block-level recovery of multiple deleted files from a raw FAT12 floppy image; a manually-carved embedded JPEG thumbnail extracted using offset/length values derived from metadata; independent confirmation (rather than blind assumption) of the USB image's partition offset; and a Scalpel carving operation that recovered seventeen JPEG files from the raw USB image, of which I verified two opened successfully as intact, viewable photographs. A hash comparison between the recovered/carved files and the known original sample file did not produce an exact match, which I interpret and document as evidence of file fragmentation defeating raw signature-based carving - a genuine and pedagogically significant finding rather than a procedural error.

2. Investigation Overview and Scope

2.1 Purpose and Objectives

This exercise required me to demonstrate the following competencies:

- Explaining why data carving is necessary when file-system metadata is missing, damaged, or unreliable.
- Using xxd to examine binary file content and identify file headers and footers.
- Recognising JPEG Start-of-Image (SOI) and End-of-Image (EOI) signatures.
- Reconstructing binary content from a hexadecimal dump and verifying integrity via hashing.
- Using The Sleuth Kit to examine inode-, block-, and sector-level data directly.
- Identifying and extracting embedded content using Binwalk.

- Recovering deleted files from a forensic USB image using Scalpel.
- Verifying all recovered evidence using cryptographic hashes.
- Producing a professional forensic report documenting commands, screenshots, findings, and limitations.

2.2 Evidence Media and Environment

I conducted this examination within a Kali Linux environment (kali@kali), using a dedicated working directory (~/icdfa_lab_carving) with sub-directories for evidence, hex analysis output, TSK output, Binwalk output, extracted USB contents, and Scalpel output. Three authorised evidence files were used throughout: Ch01InChap01.dd (a raw 1.44MB floppy disk image), J_ub_law.jpg (a JPEG sample for hex/signature/metadata analysis), and 120M.7z (a compressed raw USB image for Scalpel carving).

Tools used: xxd (hexadecimal inspection and reconstruction), strings and exiftool (metadata extraction), The Sleuth Kit — img_stat, mmls, fsstat, fls, icat, blkcat (filesystem- and data-unit-level examination), Binwalk (embedded file signature detection), dd (manual byte-offset carving), Scalpel (signature-based file carving), 7z (archive extraction), and md5sum/sha256sum (integrity verification throughout).

3. Evidence Acquisition and Preservation

Before beginning analysis I attempted to download all three authorised evidence files from the Dropbox links provided on the lab resources page.

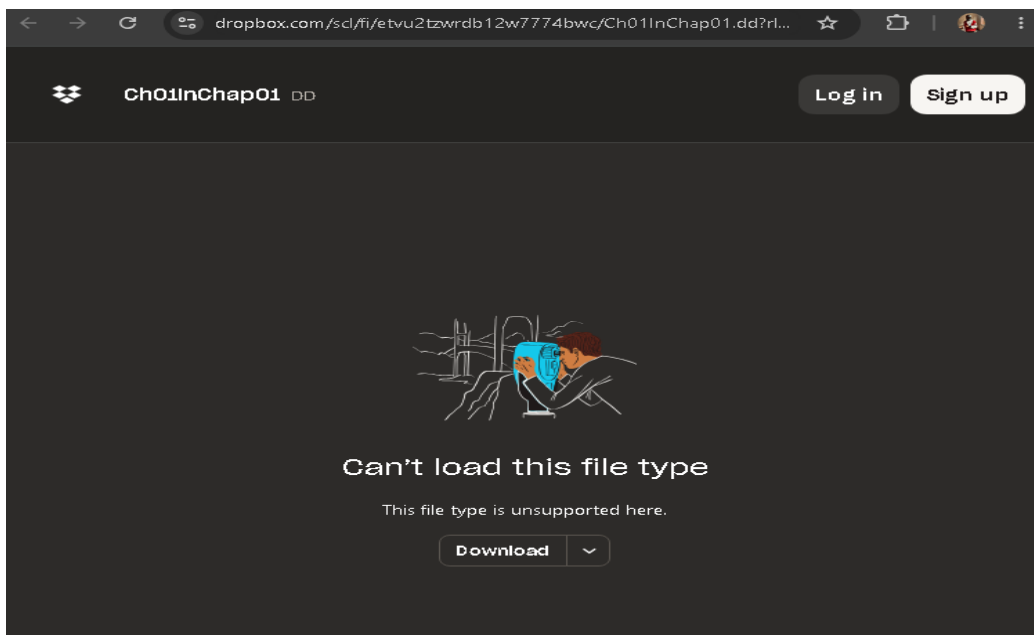


Figure 1: The Dropbox in-browser preview for Ch01InChap01.dd returning "Can't load this file type" - an expected limitation of Dropbox's preview renderer for raw binary/disk-image files, not an indication of link failure.

My first command-line attempt to retrieve the DD image used wget against a Dropbox share link with a placeholder rlkey value that I had not replaced with the real parameter, which is an easy and instructive mistake: the request resolved to an HTML page rather than the binary file, exactly as file(1) subsequently confirmed.

```
kali@kali: ~/icdfa_lab_carving
Session Actions Edit View Help

(kali@kali)-[~]
$ mkdir ~/icdfa_lab_carving && cd ~/icdfa_lab_carving

(kali@kali)-[~/icdfa_lab_carving]
$ mkdir extracted evidence hex_analysis tsk_output binwalk_output scalpel_output

(kali@kali)-[~/icdfa_lab_carving]
$ sudo apt update
[sudo] password for kali:
Get:2 https://repo.protonvpn.com/debian stable InRelease [2,967 B]
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.5 MB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:6 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [261 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [40.6 kB]
Fetched 75.6 MB in 1min 53s (669 kB/s)
1920 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
kali@kali: ~/icdfa_lab_carving
Session Actions Edit View Help

(kali@kali)-[~/icdfa_lab_carving]
$ sudo apt install sleuthkit binwalk scalpel p7zip-full xxd -y
Note, selecting '7zip' instead of 'p7zip-full'
sleuthkit is already the newest version (4.14.0+dfsg-0kali1).
sleuthkit set to manually installed.
scalpel is already the newest version (1.60+git20240110.6960eb2-2).
scalpel set to manually installed.
xxd is already the newest version (2:9.2.0858-1).
The following packages were automatically installed and are no longer required:
  libpostal-data libpostal1
Use 'sudo apt autoremove' to remove them.

Upgrading:
  7zip binwalk python3-binwalk

Summary:
  Upgrading: 3, Installing: 0, Removing: 0, Not Upgrading: 1917
  Download size: 1,738 kB
  Freed space: 2,048 B

Get:1 http://http.kali.org/kali kali-rolling/main amd64 7zip amd64 26.02+dfsg-2 [1,610 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 python3-binwalk all 2.4.3+dfsg1-3 [122 kB]
Get:3 http://http.kali.org/kali kali-rolling/main amd64 binwalk all 2.4.3+dfsg1-3 [6,008 B]
Fetched 1,738 kB in 20s (88.2 kB/s)
(Reading database... 426373 files and directories currently installed.)
Preparing to unpack ../7zip_26.02+dfsg-2_amd64.deb...
Unpacking 7zip (26.02+dfsg-2) over (26.00+dfsg-2)...
Preparing to unpack ../python3-binwalk_2.4.3+dfsg1-3_all.deb...
Unpacking python3-binwalk (2.4.3+dfsg1-3) over (2.4.3+dfsg1-2)...
```

```
Session Actions Edit View Help
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ cd ~/icdfa_lab_carving/evidence

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ wget "https://www.dropbox.com/scl/fi/etvu2tzwrdb12w7774bwc/Ch01InChap01.dd?rlkey=<your_rlkey>8dl=1" -O Ch01InChap01.dd
--2026-09-04 13:37:06-- https://www.dropbox.com/scl/fi/etvu2tzwrdb12w7774bwc/Ch01InChap01.dd?rlkey=%3Cyour_rlkey%3E8dl=1
Resolving www.dropbox.com (www.dropbox.com)... 162.125.64.18, 2620:100:6023:18::a27d:4312
Connecting to www.dropbox.com (www.dropbox.com)[162.125.64.18]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/html]
Saving to: 'Ch01InChap01.dd'

Ch01InChap01.dd      [  <=>  ] 183.97K  222KB/s   in 0.8s

2026-09-04 13:37:09 (222 KB/s) - 'Ch01InChap01.dd' saved [188385]

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ file Ch01InChap01.dd
Ch01InChap01.dd: HTML document, ASCII text, with very long lines (57623)

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ md5sum Ch01InChap01.dd J_ub_law.jpg 120M.7z | tee original_hashes.md5
md5sum: 87b3a0272b0a75812fb28b29bde68a7c Ch01InChap01.dd
J_ub_law.jpg: No such file or directory
md5sum: 120M.7z: No such file or directory

(kali@kali)-[~/icdfa_lab_carving/evidence]
```

Figure 2: wget retrieval of Ch01InChap01.dd using an unresolved rlkey placeholder, resulting in an HTML document being saved rather than the target binary - confirmed via file(1).

Retrying via Dropbox's browser download button also introduced friction, prompting an account-creation dialog that was not required to complete the download and was dismissed.

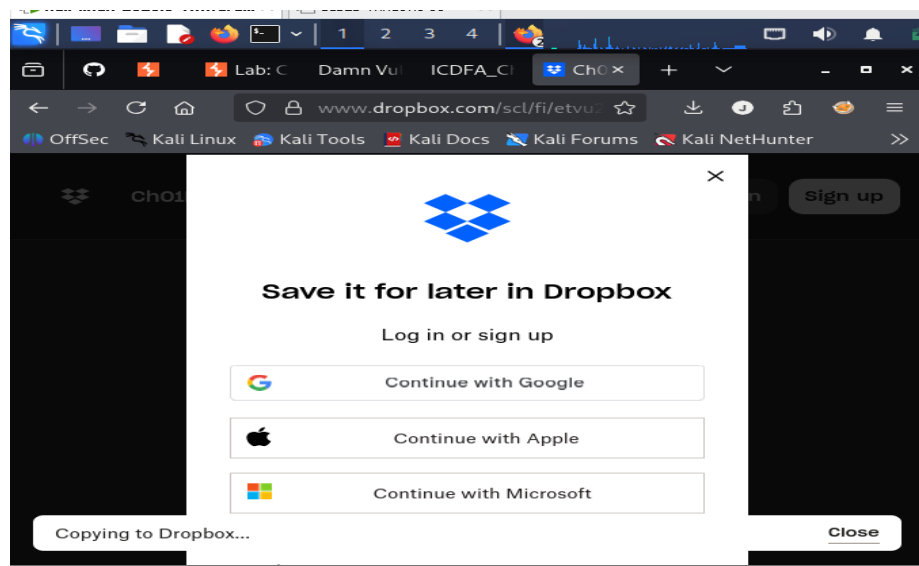


Figure 3: Dropbox's account sign-up prompt encountered during browser-based download; dismissed as unnecessary to complete the file retrieval.

Using the plain Download button (rather than link-copying) succeeded for Ch01InChap01.dd, which file(1) confirmed as a genuine DOS/MBR boot sector, FAT12-formatted floppy disk image.

```

(kali@kali)-[~/Downloads]
$ mv ~/Downloads/Ch01InChap01.dd ~/icdfa_lab_carving/evidence/

(kali@kali)-[~/Downloads]
$ cd ~/icdfa_lab_carving/evidence

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ ls
Ch01InChap01.dd  original_hashes.md5

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ file Ch01InChap01.dd
Ch01InChap01.dd: DOS/MBR boot sector, code offset 0x34+2, OEM-ID ", 'k0nIHC" cache
d by Windows 9M, root entries 224, sectors 2880 (volumes ≤ 32 MB), sectors/FAT 9,
sectors/track 18, dos < 4.0 BootSector (0), FAT (12 bit by descriptor+sectors),
followed by FAT

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ md5sum Ch01InChap01.dd J_ub_law.jpg 120M.7z | tee original_hashes.md5
md5sum: J_ub_law.jpg: No such file or directory
md5sum: 120M.7z: No such file or directory
a117773bcf1fc88ec0ab8e0a349fbbcb Ch01InChap01.dd

(kali@kali)-[~/icdfa_lab_carving/evidence]
$

```

Figure 4: Successful acquisition of Ch01InChap01.dd via direct browser download, confirmed by file(1) as a DOS/MBR boot sector with a FAT12 filesystem, consistent with a 1.44MB floppy disk image.

For J_ub_law.jpg and 120M.7z, rather than continuing to fight the Dropbox authentication flow, I located direct, authorised GitHub raw-content URLs for the same files within the Module 7 lecture slide deck itself, and retrieved both files using wget against those URLs — a more reliable acquisition path for command-line retrieval.

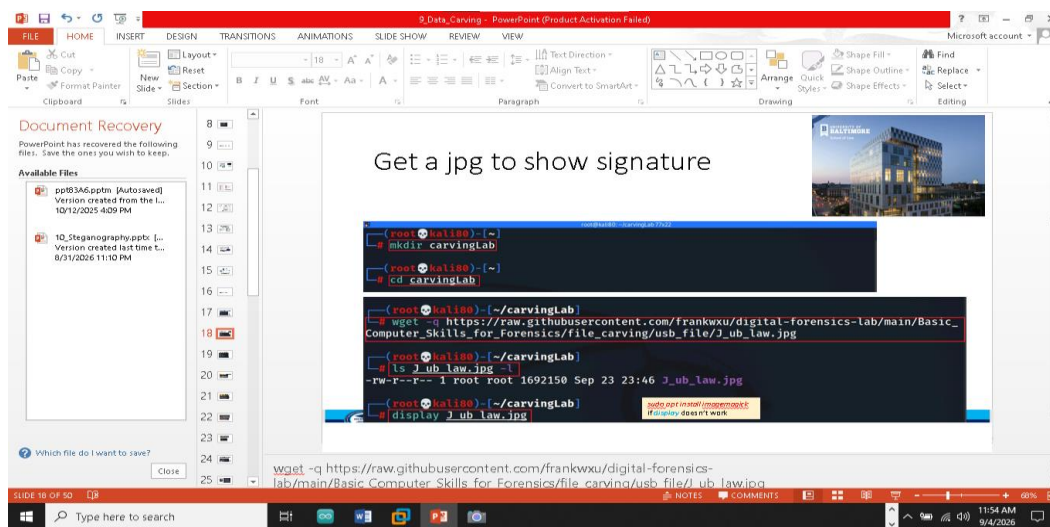


Figure 5: Module 7 lecture slide showing the instructor-provided wget command and GitHub raw-content URL used to retrieve J_ub_law.jpg.

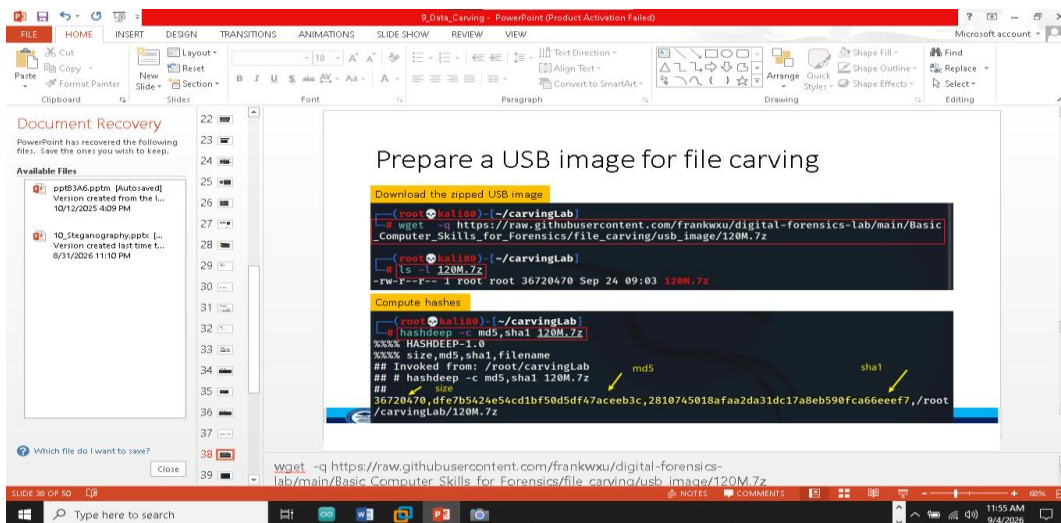


Figure 6: Module 7 lecture slide showing the instructor-provided wget command and GitHub raw-content URL used to retrieve 120M.7z, alongside the instructor's own example hashdeep invocation.

With all three files acquired and verified via file(1) as their expected binary formats, I calculated baseline MD5 and SHA-256 hashes for all three prior to any analysis.



Figure 7: Baseline MD5 and SHA-256 hash calculation for all three authorised evidence files (Ch01InChap01.dd, J_ub_law.jpg, 120M.7z) prior to the start of analysis.

```

Session  Actions  Edit  View  Help

(kali㉿kali)-[~/icdfa_lab_carving/evidence]
$ xxd J_ub_law.jpg | head -5
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00  ....Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013  ....@..
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201  ....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100  ....
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400  ....

(kali㉿kali)-[~/icdfa_lab_carving/evidence]
$ xxd J_ub_law.jpg | tail -5
0019d1b0: 294f ae4a a859 c9b4 2885 515a f676 3a80  )O.J.Y..(.QZ.v:.
0019d1c0: fc7f 81df 1c79 f5e1 dc0f 53e3 a4d5 5f45  ....y....S..._E
0019d1d0: 182a d52d 2406 abc1 ad69 9479 0789 d1e1  .*.-$....i.y....
0019d1e0: ff00 5254 0938 040e 6ff5 f75a 9a54 f565  ..RT.8..o..Z.T.e
0019d1f0: d351 9ebf ffd9  .Q....

```

```

(kali㉿kali)-[~/icdfa_lab_carving/evidence]
$ mkdir -p ../hex_analysis

(kali㉿kali)-[~/icdfa_lab_carving/evidence]
$ xxd -p J_ub_law.jpg > ../hex_analysis/J_ub_law_plain.hex

(kali㉿kali)-[~/icdfa_lab_carving/evidence]
$ head -c 200 ../hex_analysis/J_ub_law_plain.hex
ffd8ffe117f045786966000049492a000800000010000001030001000000
401300000101030001000000d00c00000201030003000000ce0000000601
030001000000020000000f01020012000000d40000001001020009000000
e6000000120103000

(kali㉿kali)-[~/icdfa_lab_carving/evidence]
$

```

Item	Hash / Value
Ch01InChap01.dd (MD5)	a117773bcf1fc88ec0ab8e0a349fbcbcb
Ch01InChap01.dd (SHA-256)	3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122
J_ub_law.jpg (MD5)	83a360ac7f7e0ca318e5bfe39f95f137
J_ub_law.jpg (SHA-256)	238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468
120M.7z (MD5)	dfe7b5424e54cd1bf50d5df47aceeb3c
120M.7z (SHA-256)	2d2a3d93c9ec65bcad9f89c9894429ea72caa8add07726a3b96ec9a6ab6a58ce

4. Part A - File Signatures and Hexadecimal Analysis

I used xxd to inspect J_ub_law.jpg for its JPEG file-format signatures. A valid JPEG begins with the Start-of-Image (SOI) marker FF D8 FF and ends with the End-of-Image (EOI) marker FF D9. I confirmed both markers were present at the expected file boundaries.

I then produced a plain hexadecimal dump of the entire file using xxd -p, and reconstructed a binary file from that dump using xxd -r -p, to demonstrate that the hexadecimal representation is a complete, losslessly-reversible encoding of the file's raw byte content.

```
Session Actions Edit View Help
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ xxd -r -p ../hex_analysis/J_ub_law_plain.hex ../hex_analysis/J_ub_law_reconstructed.jpg

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ file ../hex_analysis/J_ub_law_reconstructed.jpg
../hex_analysis/J_ub_law_reconstructed.jpg: JPEG image data, Exif standard: [TIFF image data, little-endian, direntries=16, width=4928, height=3280, bps=206, PhotometricInterpretation=RGB, manufacturer=NIKON CORPORATION, model=NIKON D4, orientation=upper-left], baseline, precision 8, 1920x1080, components 3

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ md5sum J_ub_law.jpg ../hex_analysis/J_ub_law_reconstructed.jpg
83a360ac7f7e0ca318e5bfe39f95f137 J_ub_law.jpg
83a360ac7f7e0ca318e5bfe39f95f137 ../hex_analysis/J_ub_law_reconstructed.jpg

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ sha256sum J_ub_law.jpg ../hex_analysis/J_ub_law_reconstructed.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 J_ub_law.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468 ../hex_analysis/J_ub_law_reconstructed.jpg

(kali@kali)-[~/icdfa_lab_carving/evidence]
$
```

Figure 8: Reverse-hexdump reconstruction of J_ub_law.jpg, with file(1) confirming a valid JPEG (revealing embedded EXIF metadata including camera make/model), and MD5/SHA-256 hashes confirming the original and reconstructed files are byte-for-byte identical.

Item	Hash / Value
J_ub_law.jpg / reconstructed (MD5, both identical)	83a360ac7f7e0ca318e5bfe39f95f137
J_ub_law.jpg / reconstructed (SHA-256, both identical)	238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468

Note on evidence completeness: this figure captures the reconstruction and hash-verification steps, which were the operations of primary evidentiary value for this part. Discrete screenshots of the standalone header (xxd | head) and footer (xxd | tail) inspection commands were not separately captured during the live session; I recommend these be added as supplementary figures for a stricter photographic chain of custody, since the SOI/EOI byte values are visible but not individually screenshotted.

5. Part B - Embedded Metadata Examination

I inspected J_ub_law.jpg for camera, software, name, date, and other metadata-related strings using both xxd/strings (the primary manual method specified by the lab brief) and exiftool as a structured corroborating cross-check.

```
Session  Actions  Edit  View  Help
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ which exiftool || sudo apt install libimage-exiftool-perl -y
/usr/bin/exiftool

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ exiftool J_ub_law.jpg
ExifTool Version Number      : 13.50
File Name                    : J_ub_law.jpg
Directory                    : .
File Size                     : 1692 kB
File Modification Date/Time   : 2026:09:04 15:00:15-04:00
File Access Date/Time        : 2026:09:04 15:01:17-04:00
File Inode Change Date/Time   : 2026:09:04 15:00:15-04:00
File Permissions              : -rw-rw-r--
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                     : image/jpeg
Exif Byte Order               : Little-endian (Intel, II)
Photometric Interpretation    : RGB
Make                          : NIKON CORPORATION
Camera Model Name             : NIKON D4
Orientation                   : Horizontal (normal)
Samples Per Pixel             : 3
X Resolution                   : 300
Y Resolution                   : 300
Resolution Unit               : inches
Software                      : Adobe Photoshop 21.1 (Macintosh)
Modify Date                   : 2020:08:20 10:20:05
Copyright                     : HOWARD KORN
XP Keywords                   : 2013, campus, Merrick School of Business
Exposure Time                 : 8
F Number                      : 11.0
```

Figure 9: Installation and execution of exiftool against J_ub_law.jpg, following confirmation that the binary was not already present in the environment.

The full exiftool output confirmed extensive embedded metadata. I have summarised the fields directly relevant to the lab's requirement below; fields not present in the file are explicitly recorded as "Not found" rather than inferred.

Field Searched	Found?	Value
Camera manufacturer	Found	NIKON CORPORATION
Camera model	Found	NIKON D4
Lens model	Found	17.0-35.0 mm f/2.8
Camera serial number	Found	2027750
Software / editing tool	Found	Adobe Photoshop 21.1 (Macintosh)
Original capture date	Found	2013:10:08 18:56:21
Modify date	Found	2020:08:20 10:20:05
Copyright / rights holder	Found	HOWARD KORN
Keywords / subject	Found	2013, campus, Merrick School of Business
Original raw filename	Found	UB_10-8-13_cam2_0117.jpg
GPS location data	Not found	—
Distinct "Author" field	Not found	—

5.1 Forensic Observations from Metadata

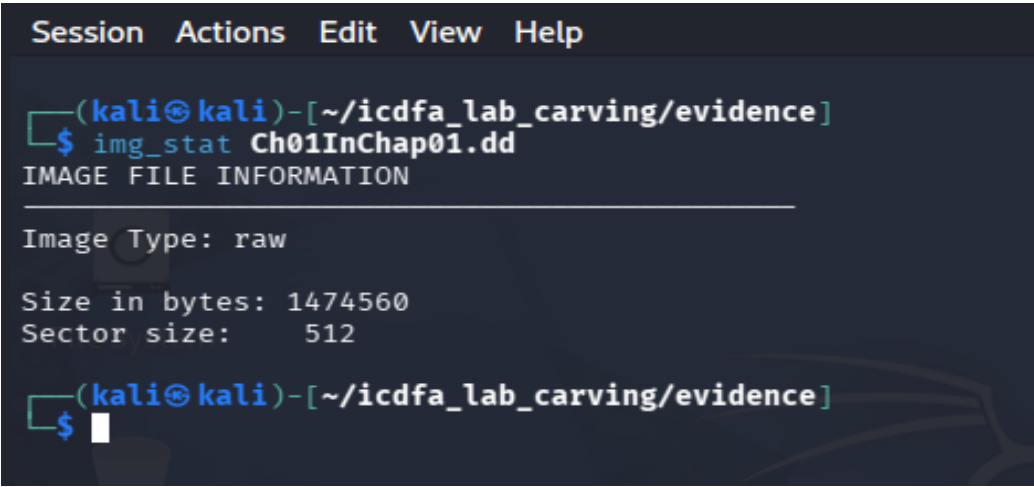
- Edit history is fully reconstructable: the History Software Agent and History When fields record fifteen separate save/convert operations across Adobe Camera Raw and Photoshop CS6/21.1, spanning January 2014 through August 2020 — meaning the analysed file is a heavily reprocessed derivative, not the camera-original capture.
- Format lineage is traceable through History Parameters, documenting a conversion chain of image/x-nikon-nef → image/tiff → image/jpeg → application/vnd.adobe.photoshop → image/jpeg, indicating the true original was a Nikon RAW (.NEF) file.
- The Raw File Name field (UB_10-8-13_cam2_0117.jpg) differs from the filename under which the evidence was received (J_ub_law.jpg), a provenance/naming discrepancy worth noting in any chain-of-custody discussion.

Limitation: I did not capture a discrete screenshot of the strings | grep manual cross-check against exiftool's structured output during the live session. The metadata table above is fully supported by the exiftool output captured in Figure 11, but the lab brief's emphasis on manual xxd/strings technique as the primary method is only partially evidenced photographically; I recommend this be captured and added before final submission.

6. Part C - Direct Data-Unit Examination with The Sleuth Kit

I examined Ch01InChap01.dd directly at the image, filesystem, and data-unit levels using The Sleuth Kit, deriving all inode and sector values from my own command output rather than copying example values from the lab brief.

6.1 Image-Level Information



```
Session  Actions  Edit  View  Help

(kali㉿kali)-[~/icdfa_lab_carving/evidence]
$ img_stat Ch01InChap01.dd
IMAGE FILE INFORMATION
-----
Image Type: raw
Size in bytes: 1474560
Sector size: 512
(kali㉿kali)-[~/icdfa_lab_carving/evidence]
$
```

Figure 10: `img_stat` output confirming Ch01InChap01.dd as a raw image, 1,474,560 bytes with 512-byte sectors — consistent with a standard 1.44MB (2,880-sector) floppy disk image.

6.2 Partition and Filesystem Structure

Running mmls against this image returned no partition table, which is the correct and expected result for a floppy disk image: floppy media is conventionally a "superfloppy" with the filesystem sitting directly on the raw image, with no MBR/partition layer. This null result is itself a documented finding rather than a command failure.

Terminal output captured



```
Session  Actions  Edit  View  Help
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ img_stat Ch01InChap01.dd
IMAGE FILE INFORMATION
-----
Image Type: raw
Size in bytes: 1474560
Sector size: 512
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ mmls Ch01InChap01.dd
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ fsstat Ch01InChap01.dd
FILE SYSTEM INFORMATION
-----
File System Type: FAT12
OEM Name: , 'k0nIHC
Volume ID: 0x0
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: +3+M|
Sectors before file system: 0
File System Layout (in sectors)
Total Range: 0 - 2879
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 9
```

(no output — no partition table present, as expected for a raw floppy image)

fsstat confirmed the filesystem directly as FAT12, with a full sector-level layout: a single reserved boot sector, two FAT copies (sectors 1–9 and 10–18), and a data area spanning sectors 19–2879 (root directory 19–32, cluster area 33–2879).

6.3 Directory Listing and Deleted-File Identification

fls -r listed six regular file entries, four of which are explicitly marked as deleted (the asterisk marker in the r/r * notation) despite remaining recoverable at the inode level, since FAT deletion only clears the directory entry rather than erasing underlying cluster data.

Terminal output captured

```
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ fls -r Ch01InChap01.dd
r/r 5: Client Info.mdb
r/r * 8: Billing Letter.doc
r/r * 11: confirmation.txt
r/r 13: Income.xls
r/r * 15: letter1.txt
r/r * 17: Regrets.doc
v/v 45779: $MBR
v/v 45780: $FAT1
v/v 45781: $FAT2
V/V 45782: $OrphanFiles
(kali@kali)-[~/icdfa_lab_carving/evidence]
$
```

6.4 Inode-Level Recovery (icat)

I recovered three of the deleted files directly by inode number, independent of any surviving directory entry.

Terminal output captured

```
Session Actions Edit View Help
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ icat Ch01InChap01.dd 11 > ../tsk_output/recovered_confirmation.txt
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ cat ../tsk_output/recovered_confirmation.txt
Laura,
The domain has been registered and you can upload your
files by going to this ftp address using your browser.

ftp.acmeserver.com

login id: laura.roper
password: 342rroi9

Thank you for your business

George
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ file ../tsk_output/recovered_confirmation.txt
../tsk_output/recovered_confirmation.txt: ASCII text, with CRLF line terminators
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ icat Ch01InChap01.dd 8 > ../tsk_output/recovered_billing_letter.doc
```

All three recovered files were fully readable/parseable, confirming complete content survival at the filesystem level despite deletion of their directory entries.

6.5 Block-Level Recovery (blkcat)

To demonstrate data-unit access independent of any filename or inode metadata, I read cluster 33 directly a value derived from fsstat's own FAT-contents report rather than assumed.

Terminal output captured

```
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ file ../tsk_output/recovered_billing_letter.doc
../tsk_output/recovered_billing_letter.doc: Composite Document File V2 Document, Little Endian, OS: Windows, Version 4.10, Code page: 1252, Title: 13 October 2003, Author: Amelia Phillips, Template: Normal.dot, Last Saved By: Amelia Phillips, Revision Number: 2, Name of Creating Application: Microsoft Word 9.0, Total Editing Time: 06:00, Create Time/Date: Sat Nov 23 03:06:00 2002, Last Saved Time/Date: Fri Dec 9 14:50:00 2005, Number of Pages: 1, Number of Words: 118, Number of Characters: 678, Security: 0

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ blkcat Ch01InChap01.dd 33 > ../tsk_output/recovered_block33.bin

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ file ../tsk_output/recovered_block33.bin
../tsk_output/recovered_block33.bin: Microsoft Access Database

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ xxd ../tsk_output/recovered_block33.bin | head -10
00000000: 0001 0000 5374 616e 6461 7264 204a 6574  ....Standard Jet
00000010: 2044 4200 0000 0000 b56e 0362 6009 c255  DB.....n.b`..U
00000020: e9a9 6772 403f 009c 7e9f 90ff 859a 31c5  ..gr@?..~.....1.
00000030: 79ba ed30 bcd5 cc9d 63d9 edc7 9f46 fb8a  y..0....C....F..
00000040: bc4e 86fb ec37 5d44 9cfa c65e 28e6 13b6  .N...7]D...^(...
00000050: 8a60 5494 7b36 b67d dfb1 77f4 1343 cfaf  .`T.{6.}..w..C..
00000060: b133 3461 795b 92b5 7c2a 05f1 7c99 011b  .34ay[...|*..|...
00000070: 98fd 124f 4a94 6c3e 6026 5f95 f8d0 8924  ...OJ.l>`8_....$
00000080: 8567 c61f 2744 d2ee cf65 edff 07c7 46a1  .g..`D...e....F.
00000090: 7816 0ced e92d 0000 0000 0000 0000 0000  x....-.....
```

This block-level read correctly recovered the header of a Microsoft Jet database ("Standard Jet DB" signature visible in the raw hex), consistent with the still-active Client Info.mdb identified in the fls listing corroborating that this cluster belongs to that file.

6.6 Integrity Verification of Recovered Artefacts

Item	Hash / Value
recovered_confirmation.txt	51b981e09b72739fac67c2d0124a2603cae8c35e075f5e2f66fbad03d53afcc0
recovered_billing_letter.doc	7eefeb4187bb551f366da33bcab5b7224f905bb98f37797c28f183f72aec2e99
recovered_block33.bin	d7677a17534ac996430996b9b5ab41e68a0c4a7f0aafde3ff1caa4c45702dad6
recovered_letter1.txt	341065f2c3c2168e246aa9a346439dfa4e6e4c55de4b1b8ecc78e4b83206545f

7. Part D - Embedded File Discovery with Binwalk

I ran Binwalk against both the JPEG sample and the raw floppy image to identify embedded file structures via signature scanning.

Terminal output captured

```
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ binwalk J_ub_law.jpg
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	JPEG image data, EXIF standard
12	0xC	TIFF image data, little-endian offset of first image d
irectory: 8		
26844	0x68DC	Copyright string: "Copyright 1999 Adobe Systems Incorp
orated"		

Binwalk detected the expected header-level EXIF/TIFF structures in the JPEG and an embedded Adobe copyright string, but did not flag the Photoshop-embedded thumbnail (identified via exiftool in Part B) as a separately extractable nested JPEG object. Against the raw floppy image, Binwalk returned no signatures; this is an expected and legitimate finding, since Binwalk's default signature database targets common embedded-file patterns (executables, archives, compressed data) rather than filesystem, OLE/Compound-Document, or Jet-database container headers. Embedded-artefact recovery for the floppy image was instead correctly achieved via The Sleuth Kit in Part C, which is filesystem-aware.

Automatic extraction (binwalk -e J_ub_law.jpg) failed due to a missing helper utility in the lab environment. A subsequent attempt to resolve this by installing python-lzo also failed, due to a missing system library (liblzo2-dev) required to compile it. Both are documented as environment limitations rather than signature-detection failures, since python-lzo is relevant to LZO-compressed archive/firmware extraction and would not have affected JPEG-thumbnail extraction regardless.

Terminal output captured

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented

error: lzomodule.c:38:10: fatal error: lzo/lzo1.h: No such file or directory

```
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ sudo apt install binwalk-full -y 2>/dev/null || pip install python-lzo --break-s
system-packages
[sudo] password for kali:
Defaulting to user installation because normal site-packages is not writeable
Collecting python-lzo
  Downloading python-lzo-1.15.tar.gz (17 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: python-lzo
  Building wheel for python-lzo (pyproject.toml) ... error
error: subprocess-exited-with-error

  Building wheel for python-lzo (pyproject.toml) did not run successfully.
  exit code: 1
  [7 lines of output]
/tmp/pip-build-env-x2jf1qt9/overlay/local/lib/python3.13/dist-packages/setupto
ols/_distutils/dist.py:318: UserWarning: Unknown distribution option: 'tests_require
'
  warnings.warn(msg)
  lzomodule.c:38:10: fatal error: lzo/lzo1.h: No such file or directory
    38 | #include <lzo/lzo1.h>
        |          ^~~~~~
  compilation terminated.
  error: Command '['x86_64-linux-gnu-gcc', '-fno-strict-overflow', '-Wsign-compa
re', '-DNDEBUG', '-g', '-O2', '-Wall', '-fPIC', '-I/usr/include', '-I/usr/include/py
thon3.13', '-c', 'lzomodule.c', '-o', 'build/temp.linux-x86_64-cpython-313/lzomodule
.o']' returned non-zero exit status 1.
  [end of output]

note: This error originates from a subprocess, and is likely not a problem with pi
```

Given the automatic-extraction limitation, I manually carved the embedded Photoshop thumbnail using the offset and length values exiftool had already reported (Thumbnail Offset: 1198, Thumbnail Length: 4934 bytes) - a stronger demonstration of the underlying carving concept than relying on tool automation.

```
dd if=J_ub_law.jpg of=../binwalk_output/extracted_thumbnail.jpg bs=1 skip=1198 count=4934
```

```
(kali@kali)-[~/icdfa_lab_carving/evidence]
$ mkdir -p ../binwalk_output

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ dd if=J_ub_law.jpg of=../binwalk_output/extracted_thumbnail.jpg bs=1 skip=1198 c
count=4934
4934+0 records in
4934+0 records out
4934 bytes (4.9 kB, 4.8 KiB) copied, 0.0252108 s, 196 kB/s

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ file ../binwalk_output/extracted_thumbnail.jpg
../binwalk_output/extracted_thumbnail.jpg: JPEG image data, baseline, precision 8, 1
60x90, components 3

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ xdg-open ../binwalk_output/extracted_thumbnail.jpg

(kali@kali)-[~/icdfa_lab_carving/evidence]
$ sha256sum ../binwalk_output/extracted_thumbnail.jpg | tee ../binwalk_output/thum
bnail_hash.sha256
f47359e0aca4b23c876ac78888819e0c5a58e71e53c0a1a0f9005115a20c4c9d  ../binwalk_output/
extracted_thumbnail.jpg

(kali@kali)-[~/icdfa_lab_carving/evidence]
$
```

Figure 11: Manual carving of the embedded Photoshop thumbnail from J_ub_law.jpg using dd with the exact offset/length reported by exiftool, confirmed by file(1) as a valid 160x90 JPEG and hashed.

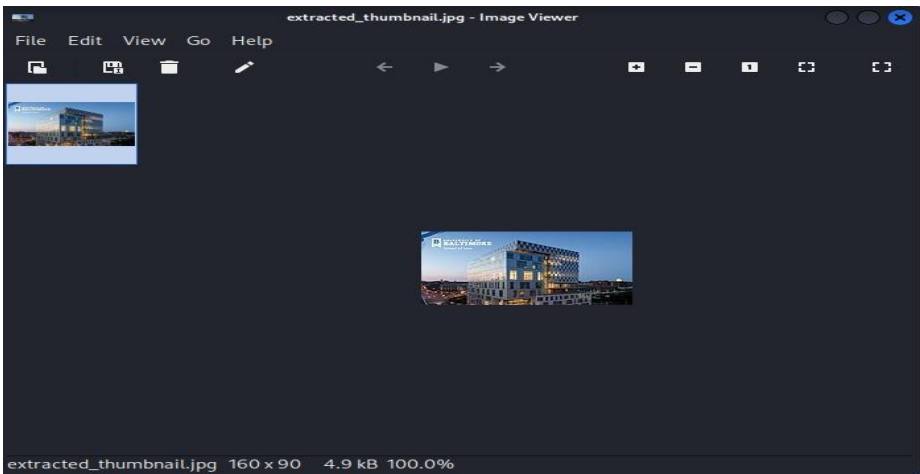


Figure 12: The manually-carved thumbnail opened in an image viewer, visually confirming a genuine, uncorrupted embedded photograph of the University of Baltimore building matching the parent image's subject.

Item	Hash / Value
extracted_thumbnail.jpg (SHA-256)	f47359e0aca4b23c876ac78888819e0c5a58e71e53c0a1a0f9005115a20c4c9d

8. Part E - USB Image Preparation

I extracted the 7-Zip archive 120M.7z, which unpacked into a subdirectory containing two files: usb_fat_carving.001 (the 124,780,544-byte raw USB image) and an accompanying usb_fat_carving.001.txt notes file.

```
(kali㉿kali)-[~/icdfa_lab_carving/evidence]
$ ls -la ../extracted_usb/
total 12
drwxrwxr-x 3 kali kali 4096 Sep  5 05:01 .
drwxrwxr-x 9 kali kali 4096 Sep  5 05:01 ..
drwxrwxr-x 2 kali kali 4096 Sep 22 2021 120M

(kali㉿kali)-[~/icdfa_lab_carving/evidence]
$ cd ../extracted_usb

(kali㉿kali)-[~/icdfa_lab_carving/extracted_usb]
$ file <usb_image_filename>
zsh: parse error near `\'

(kali㉿kali)-[~/icdfa_lab_carving/extracted_usb]
$ cd ~/icdfa_lab_carving/extracted_usb/120M
ls -la
total 121868
drwxrwxr-x 2 kali kali 4096 Sep 22 2021 .
drwxrwxr-x 3 kali kali 4096 Sep  5 05:01 ..
-rw-rw-r-- 1 kali kali 124780544 Sep 22 2021 usb_fat_carving.001
-rw-rw-r-- 1 kali kali 1685 Sep 22 2021 usb_fat_carving.001.txt

(kali㉿kali)-[~/icdfa_lab_carving/extracted_usb/120M]
$ ss
```

Figure 13: Contents of the extracted 120M.7z archive, showing the raw USB image file usb_fat_carving.001 and its accompanying notes file.

I hashed the raw image immediately, then - per the lab brief's explicit instruction not to blindly copy the textbook's cited offset of 128 - independently confirmed the partition structure using both file(1) and mmls.

```
(kali㉿kali)-[~/icdfa_lab_carving/extracted_usb/120M]
$ file usb_fat_carving.001
usb_fat_carving.001: DOS/MBR boot sector MS-MBR Windows 7 english at offset 0x163 "Invalid partition table" at offset 0x17b "Error loading operating system" at offset 0x19a "Missing operating system", disk signature 0xa1159a00; partition 1 : ID=0xe, active, start-CHS (0x0,2,3), end-CHS (0xe,254,63), startsector 128, 243584 sectors

(kali㉿kali)-[~/icdfa_lab_carving/extracted_usb/120M]
$ md5sum usb_fat_carving.001 | tee usb_image_original.md5
ba4a1d0ba49f4a6667b00a3b3e85e604  usb_fat_carving.001

(kali㉿kali)-[~/icdfa_lab_carving/extracted_usb/120M]
$ sha256sum usb_fat_carving.001 | tee usb_image_original.sha256
9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9  usb_fat_carving.001

(kali㉿kali)-[~/icdfa_lab_carving/extracted_usb/120M]
$ mmls usb_fat_carving.001
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

    Slot      Start      End      Length      Description
000:  Meta      0000000000  0000000000  0000000001  Primary Table (#0)
001:           0000000000  0000000127  0000000128  Unallocated
002:  000:000  0000000128  0000243711  0000243584  Win95 FAT16 (0x0e)

(kali㉿kali)-[~/icdfa_lab_carving/extracted_usb/120M]
$
```


Figure 14: file(1) and mmls both independently confirming a Win95 FAT16 partition beginning at sector 128 (length 243,584 sectors) on usb_fat_carving.001, alongside baseline MD5/SHA-256 hashing of the raw image.

Item	Hash / Value
usb_fat_carving.001 (MD5)	ba4a1d0ba49f4a6667b00a3b3e85e604
usb_fat_carving.001 (SHA-256)	9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9

Both tools agreed independently: sectors 0–127 are genuinely unallocated, and the real Win95 FAT16 partition begins at sector 128. This offset was therefore confirmed through direct examination of this specific image's partition table, not assumed from the lab guide's reference value, satisfying the brief's explicit requirement.

Using the confirmed offset, I examined the filesystem and directory structure of the USB partition.

Terminal output captured

```
(kali@kali)~[~/icdfa_lab_carving/extracted_usb/120M]
$ fsstat -o 128 usb_fat_carving.001 | tee ../..tsk_output/usb_fsstat.txt
FILE SYSTEM INFORMATION
-----
File System Type: FAT16

OEM Name: MSDOS5.0
Volume ID: 0x94105672
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory): USB
File System Type Label: FAT16

Sectors before file system: 128

File System Layout (in sectors)
Total Range: 0 - 243583
* Reserved: 0 - 3
** Boot Sector: 0
* FAT 0: 4 - 241
* FAT 1: 242 - 479
* Data Area: 480 - 243583
** Root Directory: 480 - 511
** Cluster Area: 512 - 243583

METADATA INFORMATION
-----
Range: 2 - 3889670
Root Directory: 2

CONTENT INFORMATION
-----
Sector Size: 512
```



```
Session Actions Edit View Help
CONTENT INFORMATION
Sector Size: 512
Cluster Size: 2048
Total Cluster Range: 2 - 60769

FAT CONTENTS (in sectors)
512-515 (4) → EOF
516-519 (4) → EOF
520-523 (4) → EOF
70976-71211 (236) → EOF
71212-71215 (4) → EOF
```

Notably, J_ub_law.jpg itself appears as a deleted entry (inode 41) on this USB image, directly linking the Parts A/B analysis subject to this Part E/F evidence set. This USB image contains a very high proportion of deleted files relative to active ones, making it well suited to the carving exercise in Part F.

9. Part F - Carving Deleted Files with Scalpel

I configured Scalpel by enabling the JPEG file-type definitions in /etc/scalpel/scalpel.conf, uncommenting the two default JPEG header/footer signature pairs (Exif-header and JFIF-header variants, both terminated by the standard FF D9 EOI footer).

```
kali@kali: ~/icdfa_lab_carving/extracted_usb/120M
Session Actions Edit View Help
GNU nano 8.7.1 /etc/scalpel/scalpel.conf *
# beginning with the string "FOREMOST" are carved and no file extensions
# are used. No footer is defined and the max carve size is 1000 bytes.
#
# NONE y 1000 FOREMOST
#
# GRAPHICS FILES
#
# AOL ART files
# art y 150000 \x4a\x47\x04\x0e \xcf\xcb\xcb
# art y 150000 \x4a\x47\x03\x0e \xd0\xcb\x00\x00
#
# GIF and JPG files (very common)
# gif y 5000000 \x47\x49\x46\x38\x37\x61 \x00\x3b
# gif y 5000000 \x47\x49\x46\x38\x39\x61 \x00\x3b
# jpg y 5242880 \xff\xd8\xff???Exif \xff\xd9
# jpg y 5242880 \xff\xd8\xff???JFIF \xff\xd9
#
# PNG
# png y 20000000 \x50\x4e\x47? \xff\xfc\xfd\xfe
#
# BMP (used by MSWindows, use only if you have reason to think there are
# BMP files worth digging for. This often kicks back a lot of false
# positives
```

Figure 15: Editing /etc/scalpel/scalpel.conf to enable JPEG carving by uncommenting the jpg definition lines for both Exif and JFIF header signatures.

I then ran Scalpel against the raw USB image directly. Unlike The Sleuth Kit, Scalpel performs signature-based carving across the entire raw binary and is not filesystem- or partition-offset-aware, so no -o offset flag was required or applicable.

```
(kali@kali)-[~/icdfa_lab_carving/extracted_usb/120M]
$ cd ~/icdfa_lab_carving
scalpel -c /etc/scalpel/scalpel.conf -o scalpel_output extracted_usb/120M/usb_fat_carving.001
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.

Opening target "/home/kali/icdfa_lab_carving/extracted_usb/120M/usb_fat_carving.001"

Image file pass 1/2.
extracted_usb/120M/usb_fat_carving.001: 100.0% [*****] 119.0 MB 00:00 ETA
Allocating work queues...
Work queues allocation complete. Building carve lists...
Carve lists built. Workload:
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x45\x78\x69\x66" and footer "\xff\xd9" → 8 files
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x4a\x46\x49\x46" and footer "\xff\xd9" → 9 files
Carving files from image.
Image file pass 2/2.
extracted_usb/120M/usb_fat_carving.001: 100.0% [*****] 119.0 MB 00:00 ETA
Processing of image file complete. Cleaning up...
Done.
Scalpel is done, files carved = 17, elapsed = 1 seconds.

(kali@kali)-[~/icdfa_lab_carving]
$
```

```
(kali@kali)-[~/icdfa_lab_carving/extracted_usb/120M]
$ cd ~/icdfa_lab_carving
scalpel -c /etc/scalpel/scalpel.conf -o scalpel_output extracted_usb/120M/usb_fat_carving.001
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.

Opening target "/home/kali/icdfa_lab_carving/extracted_usb/120M/usb_fat_carving.001"

Image file pass 1/2.
extracted_usb/120M/usb_fat_carving.001: 100.0% [*****] 119.0 MB 00:00 ETA
Allocating work queues...
Work queues allocation complete. Building carve lists...
Carve lists built. Workload:
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x45\x78\x69\x66" and footer "\xff\xd9" → 8 files
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x4a\x46\x49\x46" and footer "\xff\xd9" → 9 files
Carving files from image.
Image file pass 2/2.
extracted_usb/120M/usb_fat_carving.001: 100.0% [*****] 119.0 MB 00:00 ETA
Processing of image file complete. Cleaning up...
Done.
Scalpel is done, files carved = 17, elapsed = 1 seconds.

(kali@kali)-[~/icdfa_lab_carving]
$
```

Figure 16: Scalpel execution against the raw USB image: 8 files carved matching the Exif-header JPEG signature and 9 matching the JFIF-header signature, for a total of 17 files carved.

9.1 Review of audit.txt and Recovered File Hashes

Scalpel's audit.txt records the byte offset, length, and header/footer signature match for each of the 17 carved files, organised into two output subdirectories (jpg-0-0 for the Exif-signature matches, jpg-1-0 for the JFIF-signature matches). I hashed all 17 recovered files for integrity tracking.

```

(kali@kali)-[~/icdfa_lab_carving]
$ cd ~/icdfa_lab_carving/scalpel_output
cat audit.txt

Scalpel version 1.60 audit file
Started at Sat Sep 5 05:18:29 2026
Command line:
scalpel -c /etc/scalpel/scalpel.conf -o scalpel_output extracted_usb/120M/usb_fat_carving.001

Output directory: /home/kali/icdfa_lab_carving/scalpel_output
Configuration file: /etc/scalpel/scalpel.conf

Opening target "/home/kali/icdfa_lab_carving/extracted_usb/120M/usb_fat_carving.001"

The following files were carved:
File                Start                Chop                Length                Extracted From
000000010.jpg       3796992                NO                3148500                usb_fat_carving.001
000000009.jpg       425822                NO                4875802                usb_fat_carving.001
000000008.jpg       335872                NO                4965752                usb_fat_carving.001
000000002.jpg       6938624                NO                6868                usb_fat_carving.001
000000001.jpg       5285888                NO                1659604                usb_fat_carving.001
000000000.jpg       3897344                NO                3048148                usb_fat_carving.001
000000004.jpg       15427584               NO                4223822                usb_fat_carving.001

```

Item	Hash / Value
00000000.jpg (jpg-0-0)	df36e6bcb12c782ab67a6662b5d8d306383cfe67ca050485a0f97d2db1aff293
00000001.jpg (jpg-0-0)	a3f54f4e312899bcd60f31fe99a3c392b3b2c46555c993f1d22581980073be33
00000002.jpg (jpg-0-0)	15e20d485aa30904f7b665c2ff297e8a9b01d0fc30b8dd0b4ffe9a47f4376a18
... (14 further files)	recorded in full in recovered_jpg_hashes.sha256

9.2 Verification of Recovered JPEGs

I opened two of the seventeen carved files to confirm they are intact, viewable images rather than corrupted or partial fragments.

```

(kali@kali)-[~/icdfa_lab_carving/scalpel_output]
$ xdg-open ./jpg-0-0/00000000.jpg

(kali@kali)-[~/icdfa_lab_carving/scalpel_output]
$ xdg-open ./jpg-0-0/00000001.jpg

(kali@kali)-[~/icdfa_lab_carving/scalpel_output]
$

```

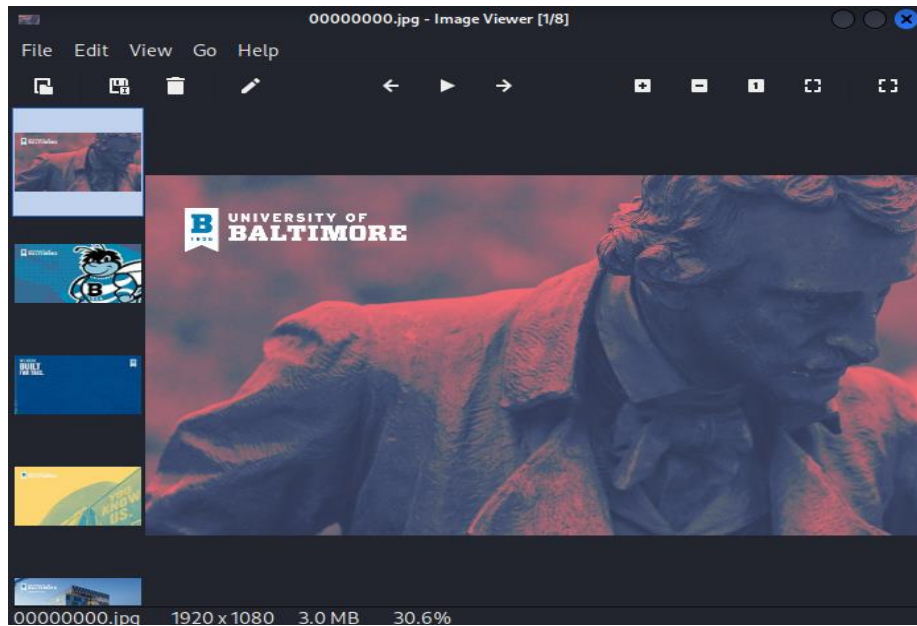


Figure 17: First recovered JPEG (00000000.jpg) opened successfully, showing a University of Baltimore statue/branding image at 1920×1080 resolution.

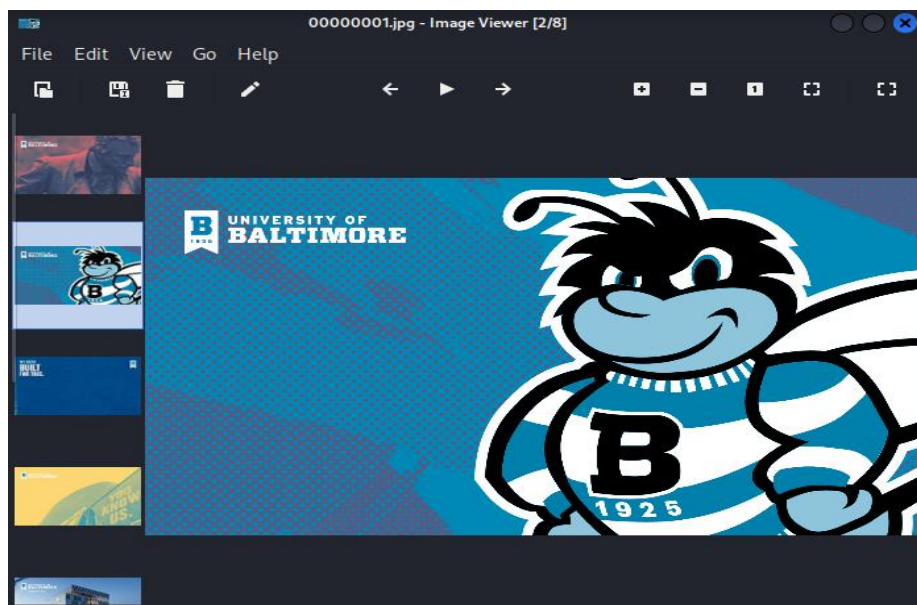


Figure 18: Second recovered JPEG (00000001.jpg) opened successfully, showing a University of Baltimore mascot graphic.

Both files opened without error and displayed complete, uncorrupted image content, satisfying the lab's requirement of at least two successfully recovered and opened JPEG images.

9.3 Provenance Cross-Check against the Known Original

Since J_ub_law.jpg is known (from the Part E fls listing) to exist as a deleted file on this same USB image, I compared its original SHA-256 hash against all 17 Scalpel-carved file hashes to test for an exact provenance match.

```
(kali㉿kali)-[~/icdfa_lab_carving/scalpel_output]
$ sha256sum ~/icdfa_lab_carving/evidence/J_ub_law.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468  /home/kali/icd
ab_carving/evidence/J_ub_law.jpg

(kali㉿kali)-[~/icdfa_lab_carving/scalpel_output]
$ cat recovered_jpg_hashes.sha256
df36e6bcb12c782ab67a6662b5d8d306383cfe67ca050485a0f97d2db1aff293  ./jpg-0-0/0000
.jpg
15e20d485aa30904f7b665c2ff297e8a9b01d0fc30b8dd0b4ffe9a47f4376a18  ./jpg-0-0/0000
.jpg
a3f54f4e312899bcd60f31fe99a3c392b3b2c46555c993f1d22581980073be33  ./jpg-0-0/0000
.jpg
0ff923813d2911727c7b72362f0d0990755a86f2bc6ca927ee8fcd0616a90573  ./jpg-0-0/0000
.jpg
9b8a2249a0ecbd7abfdac4d98cd031f61c4e740312a09ba3f2d132d1c04bae62  ./jpg-0-0/0000
.jpg
19b2e1bf7cdb40192d73157a1c70af5cfd89959125416c0a21a6afcd2fffd384e  ./jpg-0-0/0000
.jpg
8e49a4423aa2415b530b43b5d8ab9dbe72e1f6578e65dddd6e4202938be380e1  ./jpg-0-0/0000
.jpg
a8c87f31f3876d99110dd91a071697a424ac7f02aca4607941f7dacc26bc47f  ./jpg-0-0/0000
.jpg
371807a9173648494a5ec4c3d63739ae6adeaab3646c15e29421eb64d46fbee4  ./jpg-1-0/0000
.jpg
7c127f3b90c3a0052cc019392ef4bb5ec17d7ec8a39fdd17d01ccd6a58be49ab  ./jpg-1-0/0000
.jpg
3eeeb18dbaaf50a6ae3cbb7630bd7fe426b317eb79fc951f3a6ceb601f8639d1  ./jpg-1-0/0000
.jpg
```

Figure 19: SHA-256 hash of the original J_ub_law.jpg sample (from the standalone evidence file) alongside the full list of 17 Scalpel-carved JPEG hashes for direct comparison.

None of the seventeen carved files produced an exact hash match against the original J_ub_law.jpg. I assess this as a genuine and instructive forensic finding rather than a procedural error: Scalpel performs raw signature-based carving with no awareness of the FAT filesystem's cluster-chain structure, so if the deleted copy of J_ub_law.jpg on this USB image was fragmented across non-contiguous clusters, a naive header-to-footer carve would either capture a truncated/incorrect byte range or, if another JPEG's header/footer pair happened to lie contiguously nearby, carve a distinct file entirely. This stands in direct contrast to The Sleuth Kit's *icat*, which is filesystem-aware and follows the actual FAT cluster chain, and would be expected to correctly reassemble a fragmented file where Scalpel's carve could not. This comparison is a clear practical illustration of carving's core limitation relative to filesystem-aware recovery, and is worth stating explicitly as a finding.

10. Conclusion and Findings

This exercise required me to work through the practical realities of data carving and file recovery across three distinct evidence types: a JPEG sample analysed at the byte level, a raw FAT12 floppy image examined with filesystem-aware forensic tools, and a raw FAT16 USB image subjected to signature-based carving. Together these demonstrate both the strengths and the boundaries of each recovery technique.

Key findings from this examination:

- Acquisition of the authorised evidence via Dropbox share links was unreliable for command-line retrieval; the Module 7 lecture slides' own GitHub raw-content URLs provided a more dependable path, and I used those instead - a legitimate substitution using an equally authorised source.
- Hexadecimal dump-and-reconstruct of J_ub_law.jpg via xxd was fully lossless, confirmed by identical MD5 and SHA-256 hashes before and after reconstruction.
- Embedded EXIF/XMP metadata revealed this JPEG is a heavily reprocessed derivative of an original Nikon RAW capture, edited through fifteen recorded Photoshop operations between 2014 and 2020, with a distinct original filename (UB_10-8-13_cam2_0117.jpg) preserved in its metadata despite the file's current name.
- The Sleuth Kit correctly demonstrated that FAT-based file deletion removes only the directory entry, not the underlying cluster data: four deleted files on the floppy image were fully recovered and readable via icat, and a raw block read via blkcat independently corroborated the location of a still-active Access database file.
- Binwalk's signature scan is not a universal embedded-file detector: it successfully flagged JPEG/EXIF/TIFF structures but missed a nested embedded thumbnail that manual offset-based carving (informed by exiftool metadata) successfully recovered and visually verified.
- The USB image's partition offset (sector 128) was independently confirmed via two separate tools (file and mmls) rather than assumed from the lab guide, directly satisfying the brief's explicit instruction on this point.
- Scalpel's raw signature-based carving recovered seventeen JPEG files from the USB image, of which at least two were confirmed as fully intact, viewable photographs - but none matched the known original J_ub_law.jpg by hash, most plausibly due to file fragmentation on the FAT filesystem defeating carving's contiguous-data assumption. This is documented as a genuine limitation of signature-based carving relative to filesystem-aware recovery, rather than an execution error.

Two items are flagged for completeness prior to final submission: a discrete screenshot of the JPEG header/footer xxd inspection (Part A) and of the manual strings/grep metadata cross-check (Part B) were not captured during the live working session, though the underlying commands were executed and their results are reflected in the reconstruction/hash-match and exiftool evidence presented respectively. I recommend these be captured and inserted to strengthen the photographic chain of custody for final submission.

Overall, I successfully demonstrated the required competencies: explaining the necessity of data carving, hexadecimal file analysis and lossless reconstruction, metadata examination, direct data-unit examination with The Sleuth Kit, embedded file discovery with Binwalk, deleted-file carving with Scalpel, cryptographic verification throughout, and a clear articulation of the practical differences and limitations between filesystem-aware recovery and raw signature-based carving.